Homework 2

Gabriela Rubio

February 7, 2026

1. **Algorithm description:** The algorithm iterates through the arrays using two different points. First pointer will initialize at the beginning of one array, and the other pointer will initialize at the end of the other array (which array is chosen for each doesn't matter). At every iteration find the current sum when accessing the corresponding elements. If the sum is equal to the target, return the pair. If the sum is too big, decrement the second pointer. If the sum is too small, increase the first pointer. Continue until a pair is found, or until the pointers go out of bounds.

   Pseudocode:

```
Algorithm: FindPairSum
Input: Two sorter arrays A and B of size n, and a target value x
Output: A pair (A[i], B[j]) such that A[i] + B[j] = x,
or NULL if there's no such pair

i = 0
j = n - 1

while i < n and j >= 0:
    sum = A[i] + B[j]
    if sum == x:
        return A[i], B[j]
    else if sum > x:
        j = j - 1
    else
        i = i + 1

return NULL
```

   Correctness: The algorithm is correct because we eliminate impossible pairs, but not valid pairs. Since A and B are sorted, if A[i] + B[j] is bigger than x, we also know that A[i] paired with any element bigger/to the right of B[j] will also be bigger than x, so we can safely go the opposite direction. The same thing happens when A[i] + B[j] is smaller than x: then, any element to the left of A[i] will be too small for B[j]. In the case that A[i] + B[j] = x, then we immediately return (and we don't return a pair in any other scenario, so we never get an invalid pair). If no pair exists, the pointers will go out of bounds and then we return null. **Time Analysis:**

```
i = 0                          // O(1)
j = n - 1                      // O(1)

while i < n and j >= 0:        // at most 2n
    sum = A[i] + B[j]          // O(1)
    if sum = x:                // O(1)
        return A[i], B[j]      // O(1)
    else if sum > x:           // O(1)
        j = j -1               // O(1)
    else                       // O(1)
        i = j + 1              // O(1)
```

```
return NULL                 // O(1)
```

The algorithm runs in **O(n) time**. The initializations take constant time. The while loop will execute at most 2n times because each iterations either increments i OR decrements j at most n times. Then, each iteration will perform only constant-time operations. Therefore, T(n) = 2 + 6(2n) = 2 + 12n = O(n)

2. **Algorithm description:** A divide-and-conquer algorithm that recursively merges the k sorted lists. In every recursive step, it divides the k lists into two halves, recursively merge each half, then merge the two resulting sorted lists. The base case is when there's only one list, which is already sorted.

   Pseudocode:

```
Algorithm: MergeSortedLists
Input: Array L containing k sorted lists, indices i and j
Output: One sorted list containing all elements from L

if i = j:
    return L[i]

mid = (i + j)/2
list1 = MergeSortedLists(L, i, mid)
list2 = MergeSortedLists(L, mid + 1, j)
return MergeTwoLists(list1, list2)

Algorithm: MergeTwoLists
Input: Two sorted lists list1 and list2
Output: One sorted list containing all elements from both lists

result = {}

a, b = 1
n1 = length of list1
n2 = length of list2

while a <= n1 and b <= n2:
    if list1[a] <= list2[b]:
        append list1[a] to result
        a = a + 1
    else
        append list2[b] to result
        b = b + 1

while a <= n1:
    append list1[a] to result
    a = a + 1

while b <= n2
    append list2[b] to result
    b = b + 1

return result
```

Correctness: When analyzing every step of the divide-and-conquer approach, we can see that this algorithm is correct. The base case returns a single list that is already sorted (there's no smaller base case). We divide the k lists into two groups and apply the recursive steps to them so each

groups becomes one list, then merge both groups together. The combine step correctly merges by selecting the smaller element from the front of either list, given that they are already sorted, which preserves sorted order through merging. If lists differ in size, we copy the leftover amount without need to compare.

Time Analysis: The base case takes O(1) time, the recursive calls are $2T(\frac{k}{2})$, and the merge and count algorithm is $O(n)$.

The recurrence relation is: $T(k) = 2T(\frac{k}{2}) + O(n)$. We have 2 different variables so we can't use use the master theorem. We can use a recursion tree. At each level of the recursion, all n elements are merged only once, so the work per level is $O(n)$. The number of level is the number of times we half our list of lists, which would be $O(\log k)$.

The total running time is $O(n \log k)$.

3. (a) {4, 2}, {4, 1}, {2, 1}, {9, 1}, {9, 7}

   (b) An element sorted in descending order would have the most inversions. It would have $\frac{n(n-1)}{2}$ inversions.

   (c) **Algorithm Description:** The algorithm is a modified merge sort. When recursively sorting the left and right halves, also count the inversions within each half: whenever an element from the right subarray is smaller than one from the left subarray, all the elements from the left subarray are inversions for that number. We return the total count of inversions from each half and then merge.

   Pseudocode:

```
Algorithm CountInversions
Input: Array A, indices i and j
Output: Number of inversions in A, and also a sorted A

if i >= j:
    return 0

mid = (i + j)/2

leftInversions = CountInversions(A, i, mid)
rightInversions = CountInversions(A, mid + 1, j)
totalInversions = MergeAndCount(A, i, mid, j)

return leftInversions + rightInversions + totalInversions

Algorithm MergeAndCount
Input: array A, indices i, mid, and j
Output: number of inversions between the subarrays, sorted A

L = A[i...mid]
R = A[mid+1...j]
inversions = 0

a = 1
b = 1
c == i

while a <= length(L) and b <= length(R):
    if L[a] <= R[b]:
        A[c] = L[a]
        a = a + 1
    else:
        A[c] = R[b]
        inversions += length(L) - a + 1
        b = b + 1
```

```
        c = c + 1

    Copy remaining elements of L to A
    Copy remaining elements of R to A
    return inversions
```

Correctness: The algorithm is correct because it counts all of the inversions exactly once. Inversions are divided into three categories: those entirely within the left half, those entirely within the right half, and those between the two halves. The recursive calls count inversions within each half. During merging, when we select R[j] before L[i], it means R[j] ¡ L[i]. Since both subarrays are sorted, R[j] is also smaller than all remaining elements L[i], L[i+1], ..., L[m], forming exactly (m - i + 1) inversions. We add this count and continue merging. Since the array gets sorted in the process, all inversions are counted correctly without double-counting.

Time Analysis: The base case takes O(1) time, the recursive calls are $2T(\frac{n}{2})$, and the merge and count algorithm is $O(n)$.
Let's use the master theorem:
$a = 2, b = 2, f(n) = O(n)$
$f(n) = n^{\log_b a}$
$n^{\log_2 2} = n^1 = n$

This is the 2nd case of the Master Theorem, where: $f(n) = O(n) = \Theta(n) = \Theta(n^{\log_2 2})$
$f(n) = \Theta(n^{\log_b a} log^k n) \implies T(n) = \Theta(n^{\log_b a} log^{k+1} n)$
$f(n) = \Theta(n) = \Theta(n \times 1) = \Theta(n \log^0 n)$

So $k = 0$ And $T(n) = \Theta(n \log^{0+1} n) = \Theta(n \log n)$

The time complexity is **O(n log n)**.

4. (a) $T(n) = 2 \times T(\frac{n}{2}) + n^3$

Using the master theorem:
$a = 2, b = 2, f(n) = n^3$
$n^{\log_b a} = n^{\log_2 2} = n^1 = n$

This is the 3rd case, where:
IF: $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some $\epsilon > 0$
AND: $af(\frac{n}{b}) \leq c(f(n))$ for some $c < 1$
THEN: $T(n) = \Theta(f(n))$

$f(n) = n^3 = \Omega(n^{1+\epsilon})$ for $\epsilon = 2$ $2f(\frac{n}{2}) \leq c(f(n))$ for $c < 1$

Then $T(n) = \Theta(n^3)$

(b) $T(n) = 4 \times T(\frac{n}{2}) + n\sqrt{n}$

Using the master theorem:
$a = 4, b = 2, f(n) = n\sqrt{n} = n^{\frac{3}{2}}$
$n^{\log_b a} = n^{\log_2 4} = n^2$

This is the 1st case, where:
IF: $f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$ THEN: $T(n) = \Theta(n^{\log_b a})$

$f(n) = n^{\frac{3}{2}} = O(n^{2-\epsilon})$ for $\epsilon = \frac{1}{2}$

Then $T(n) = \Theta(n^2)$

(c) $T(n) = 3 \times T(\frac{n}{3}) + n \log n$

Using the master theorem:
$a = 3, b = 3, f(n) = n \log n$
$n^{\log_b a} = n^{\log_3 3} = n^1 = n$

This is the 2nd case, because even though $n \log n$ grows a bit faster than n, it's not enough for case 3. So:
IF: $f(n) = \Theta(n^{\log_b a} \log^k n)$ for some $k \geq 0$
THEN: $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$

$f(n) = n \log n = \Theta(n \log^k n)$ for $k = 1$
Then $T(n) = \Theta(n \log^2 n)$

(d) $T(n) = T(\frac{3n}{4}) + n$

Using the master theorem:
$a = 1, b = \frac{4}{3}, f(n) = n$
$n^{\log_b a} = n^{\log_{\frac{4}{3}} 1} = n^0 = 1$

This is the 3rd case, where:
IF: $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some $\epsilon > 0$
AND: $af(\frac{n}{b}) \leq c(f(n))$ for some $c < 1$
THEN: $T(n) = \Theta(f(n))$

$f(n) = n = \Omega(n^{0+\epsilon})$ for $\epsilon = 1$ $f(\frac{3n}{4}) \leq c(f(n))$ for $c < 1$

Then $T(n) = \Theta(n)$

5. **Algorithm Description:** The algorithm uses a divide-and-conquer approach, similar to the maximum subarray problem. For a given range of days we first divide the array of prices into two halves, then recursively compute the maximum profit in the left half and the right half. After this, we can Compute the maximum profit from buying in the left half and selling in the right half (which would be the max price in the right half minus the min price in the left half). The overall maximum profit is the largest of the three values above.

   Pseudocode:

```
Algorithm MaxProfit(array, i, j)
Input: array[1...n] with prices, indices i and j
Output: maxProfit, sellDay, buyDay

if i >= j
    return 0, i, j

mid = (i + j)/2

leftProfit, leftBuy, leftSell = MaxProfit(array, i, mid)
rightProfit, rightBuy, rightSell = MaxProfit(array, mid+1, j)
```

```
minLeft = index of min in the array[i...mid]
maxRight = index of max in the array[mid+1...j]
crossProfit = array[maxRight] - array[minLeft]

if crossProfit >= leftProfit AND crossProfit >= rightProfit:
    return crossProfit, minLeft, maxRight
else if leftProfit >= rightProfit:
    return leftProfit, leftBuy, leftSell
else:
    return rightProfit, rightBuy, rightSell
```

Correctness: The algorithm is correct because the base is one single day (so no profit is made), and in every recursive step there's only three possibilities: the best pair is in the left half, the best pair is in the right half, or the best pair is a cross (best buy on left, best sell on right). Since we compare the three possibilities and return whichever has the maximum profit, the actual maximum of the whole array is found.

Time Analysis: The base case takes O(1) time, the recurive calls are $2T(\frac{n}{2})$, finding the minimum and maximum takes O(n) time. The recurence then is $T(n) = 2T(\frac{n}{2}) + O(n)$.

Let's use the master theorem:

$a = 2, b = 2, f(n) = O(n)$

$f(n) = n^{\log_b a}$

$n^{\log_2 2} = n^1 = n$

This is the 2nd case of the Master Theorem, where: $f(n) = O(n) = \Theta(n) = \Theta(n^{\log_2 2})$

$f(n) = \Theta(n^{\log_b a} log^k n) \implies T(n) = \Theta(n^{\log_b a} log^{k+1} n)$

$f(n) = \Theta(n) = \Theta(n \times 1) = \Theta(n \log^0 n)$

So $k = 0$ And $T(n) = \Theta(n \log^{0+1} n) = \Theta(n \log n)$

The time complexity is **O(n log n)**.